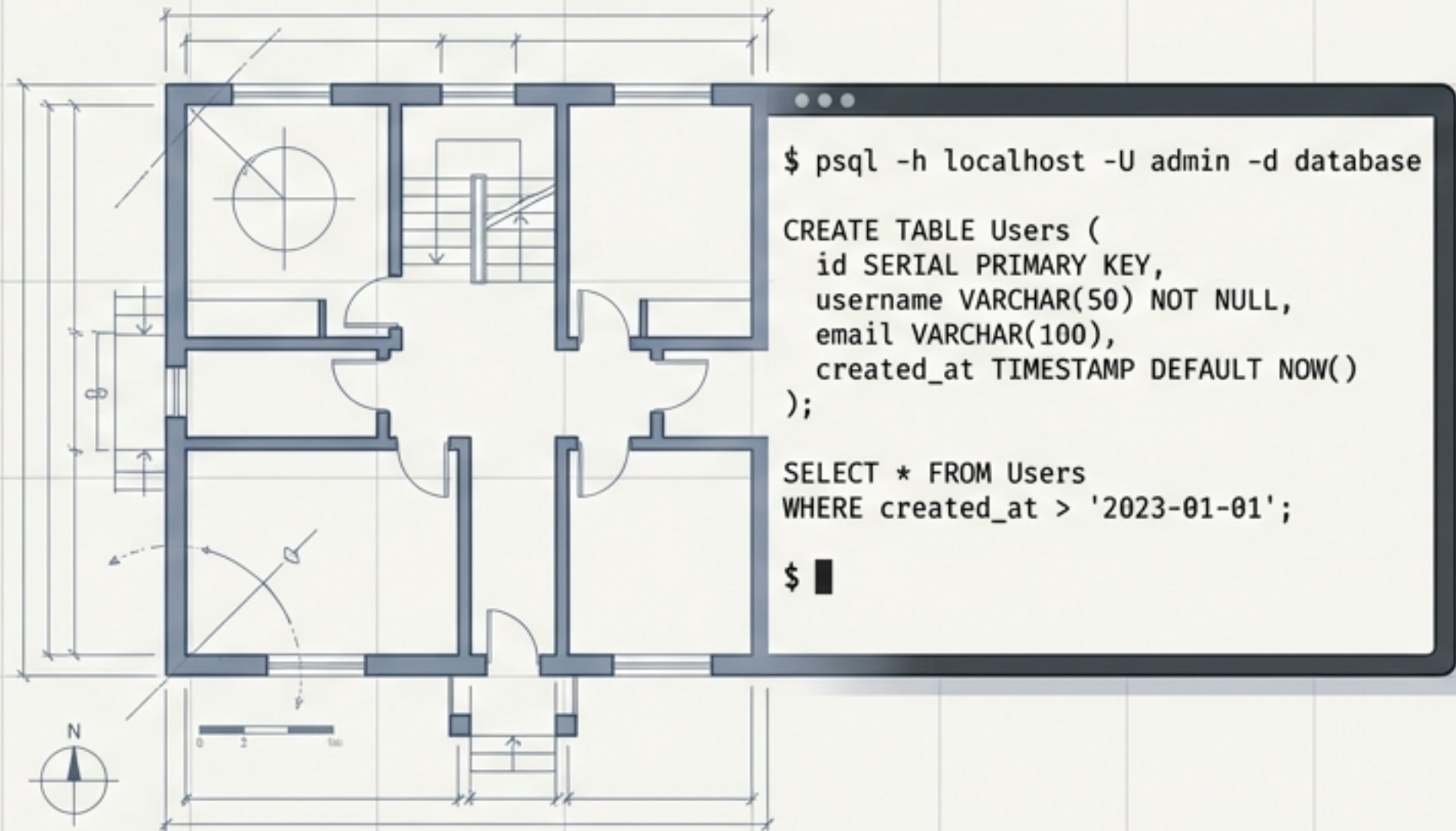
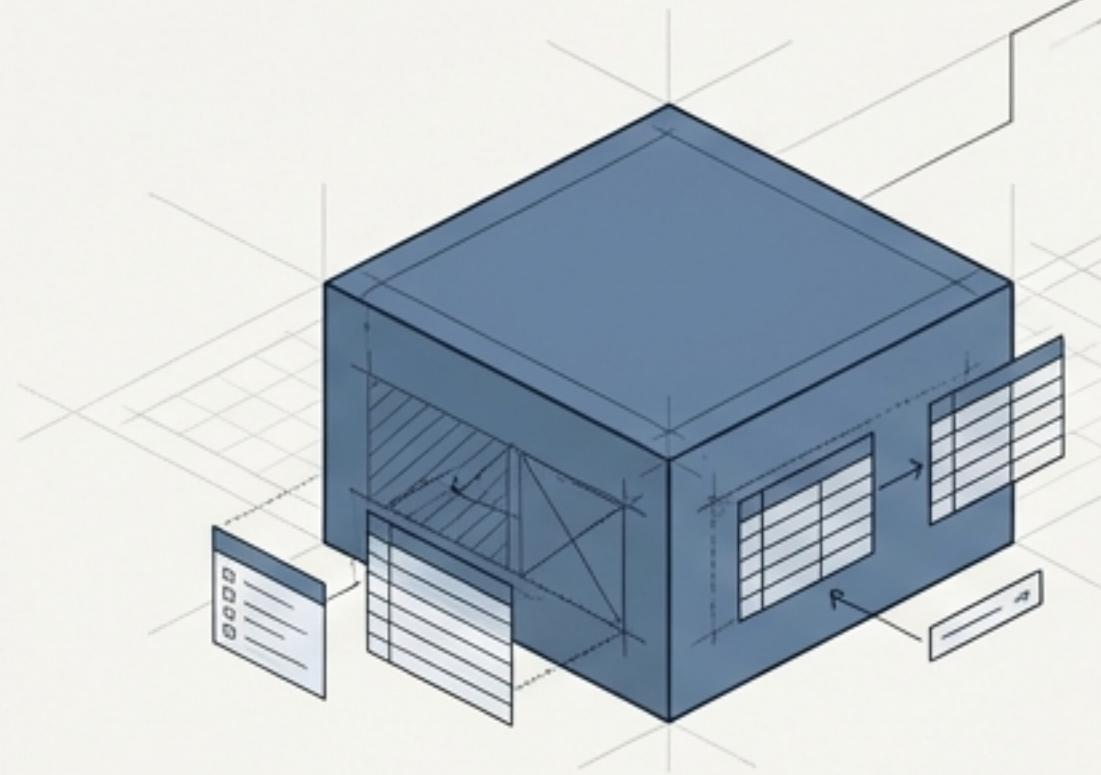


The Complete SQL Blueprint

Optionally all included but a strict grid system



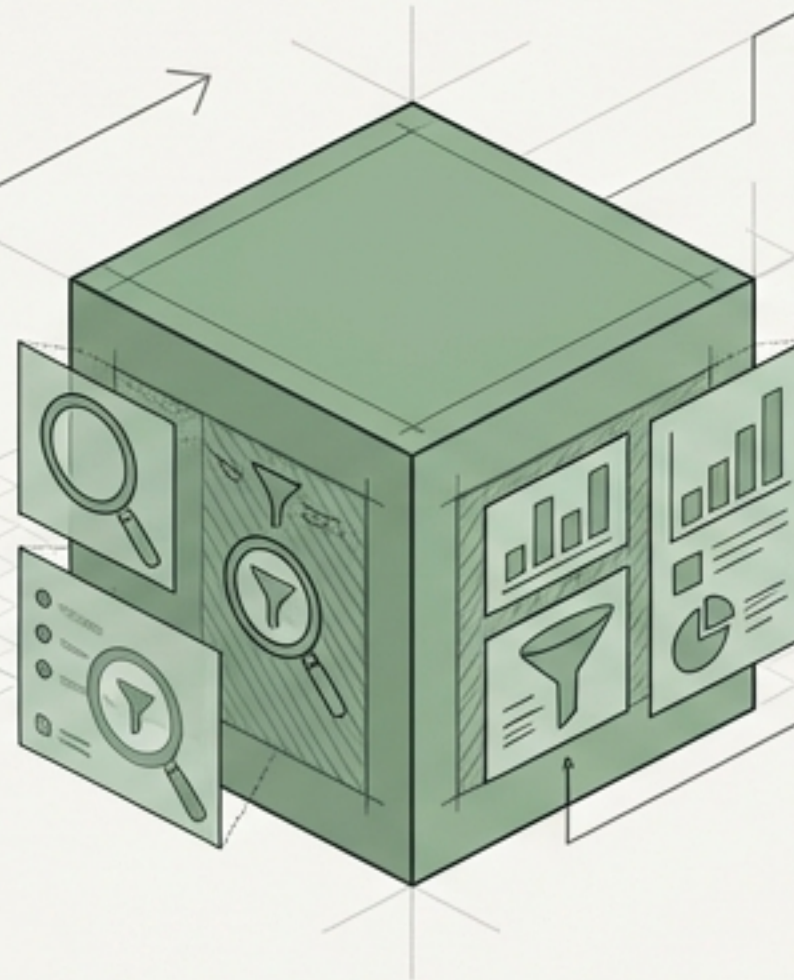
EVOLVING FROM ARCHITECT TO MASTERMIND



Phase 1: The Architect

Commands: DDL & DML

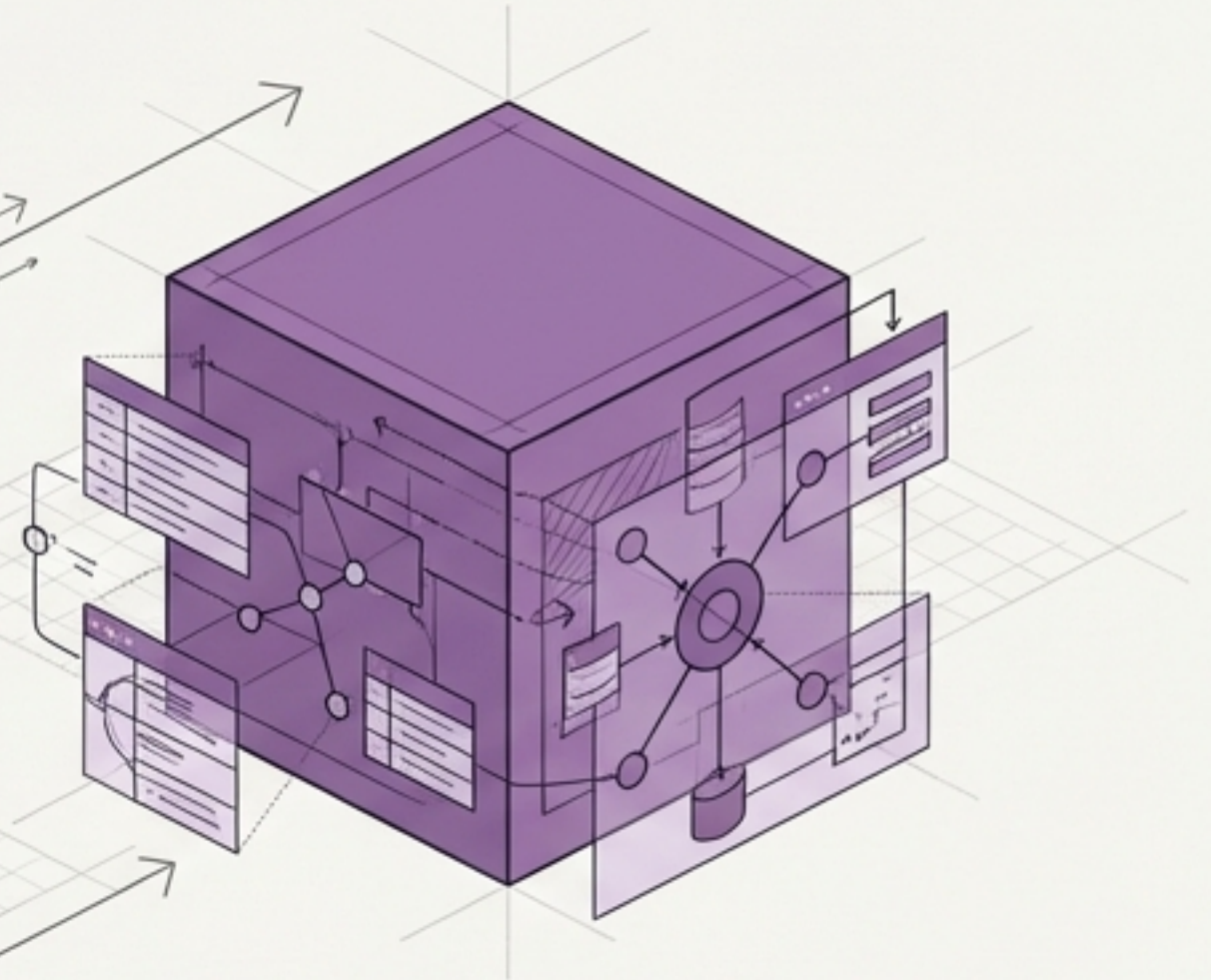
Focus: Building the foundation and populating it with data.



Phase 2: The Detective

Commands: Querying & Aggregates

Focus: Asking questions, filtering noise, and summarising insights.



Phase 3: The Mastermind

Commands: Joins & Subqueries

Focus: Connecting relational webs and performing deep-dive investigations.

Defining and optimising the database structure

CREATE

Creates a new database and objects (table, index, view, or stored procedure).

```
CREATE TABLE table_name  
(column1 datatype1,  
column2 datatype2, ...);
```

```
CREATE TABLE employees  
(employee_id INT PRIMARY KEY,  
first_name VARCHAR(50),  
last_name VARCHAR(50),  
age INT);
```

ALTER

Adds, deletes, or modifies columns in an existing table.

```
ALTER TABLE table_name  
ADD column_name datatype;
```

```
ALTER TABLE customers  
ADD email VARCHAR(100);
```

Tearing down and resetting the foundation

DROP

Drops an existing table entirely from a database.

```
DROP TABLE table_name;
```

```
DROP TABLE customers;
```

TRUNCATE

Deletes the data inside a table, but preserves the table structure itself.

```
TRUNCATE TABLE table_name;
```

```
TRUNCATE TABLE customers;
```

Populating and manipulating data rows

INSERT

Adds new records to a table.

```
INSERT INTO table_name  
(column1, column2)  
VALUES (value1, value2);
```

```
INSERT INTO customers  
(first_name, last_name)  
VALUES ('Mary', 'Doe');
```

UPDATE

Modifies existing records in a table.

```
UPDATE table_name  
SET column1 = value1,  
    column2 = value2  
WHERE condition;
```

```
UPDATE employees  
SET employee_name = 'John Doe',  
    department = 'Marketing';  
WHERE condition;
```

DELETE

Removes specific records from a table.

```
DELETE FROM table_name  
WHERE condition;
```

```
DELETE FROM employees  
WHERE employee_name =  
    'John Doe';
```

The art of filtering and ordering data

SELECT Statement

The primary command used to retrieve data from a database.

```
SELECT column1, column2  
FROM table_name;
```

```
SELECT first_name, last_name  
FROM customers;
```

WHERE Clause

Filters rows based on a specified condition.

```
SELECT *  
FROM table_name  
WHERE condition;
```

```
SELECT *  
FROM customers  
WHERE age > 30;
```

ORDER BY Clause

Sorts the result set in ascending (ASC) or descending (DESC) order.

```
SELECT *  
FROM table_name  
ORDER BY column_name ASC|DESC;
```

```
SELECT *  
FROM products  
ORDER BY price DESC;
```

Categorising insights with grouped data

GROUP BY Clause

Groups rows based on values in a specified column. Often used with aggregate functions.



```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name;
```

```
SELECT category, COUNT(*)  
FROM products  
GROUP BY category;
```

HAVING Clause

Filters grouped results based on a specified condition.



```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name  
HAVING condition;
```

```
SELECT category, COUNT(*)  
FROM products  
GROUP BY category  
HAVING COUNT(*) > 5;
```

Summarising the noise with aggregate functions

COUNT()

Counts the number of rows or non-null values in a specified column.

```
SELECT COUNT(column_name)
FROM table_name;
```

```
SELECT COUNT(age)
FROM employees;
```

SUM()

Calculates the total sum of all values in a specified column.

```
SELECT SUM(column_name)
FROM table_name;
```

```
SELECT SUM(revenue)
FROM sales;
```

AVG()

Calculates the average (mean) of all values in a specified column.

```
SELECT AVG(column_name)
FROM table_name;
```

```
SELECT AVG(price)
FROM products;
```

Finding the extremes within your data

MIN



MIN()

Returns the minimum (lowest) value in a specified column.

Blueprint

```
SELECT MIN(column_name)
FROM table_name;
```

Reality

```
SELECT MIN(price)
FROM products;
```

MAX



MAX()

Returns the maximum (highest) value in a specified column.

Blueprint

```
SELECT MAX(column_name)
FROM table_name;
```

Reality

```
SELECT MAX(price)
FROM products;
```

Connecting relational data with inner joins

INNER JOIN

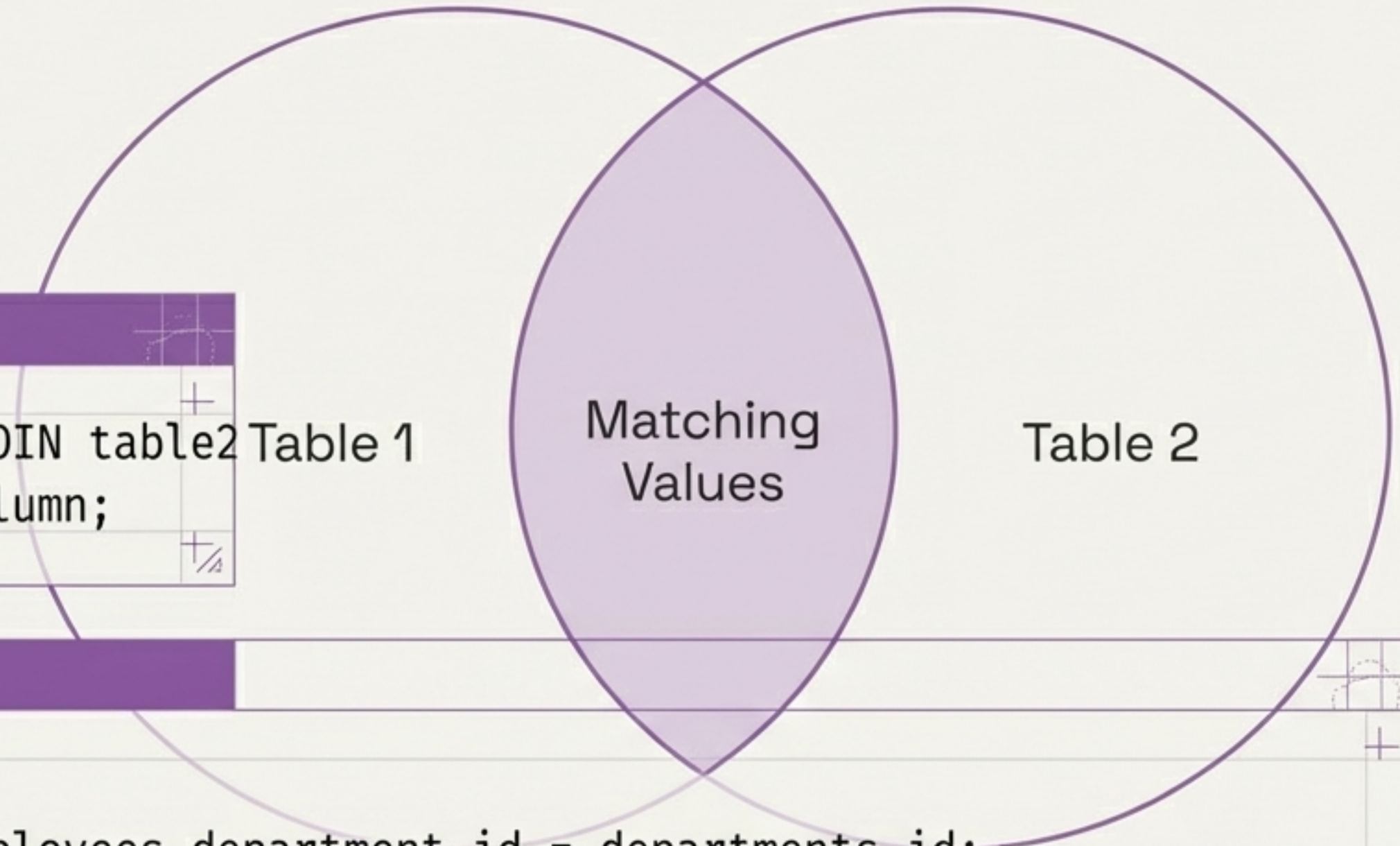
Returns rows with matching values in both tables.

Blueprint

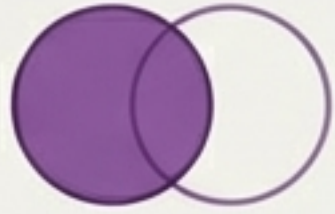
```
SELECT * FROM table1 INNER JOIN table2  
ON table1.column = table2.column;
```

Reality

```
SELECT * FROM employees  
INNER JOIN departments ON employees.department_id = departments.id;
```

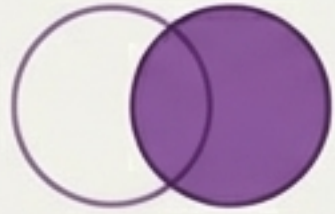


Catching missing pieces using outer joins



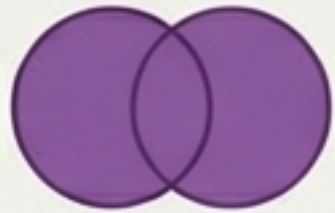
LEFT JOIN / LEFT OUTER JOIN

Returns all rows from the left table and matching rows from the right table.



RIGHT JOIN / RIGHT OUTER JOIN

Returns all rows from the right table and matching rows from the left.



FULL JOIN / FULL OUTER JOIN

Returns all rows when there is a match in either the left or right table.

Blueprint

```
SELECT * FROM table1  
LEFT JOIN table2  
ON table1.column =
```

Reality

```
SELECT * FROM employees  
LEFT JOIN departments ON  
employees.department_id =
```

Blueprint

```
SELECT * FROM table1  
RIGHT JOIN table2  
ON table1.column =
```

Reality

```
SELECT * FROM employees  
RIGHT JOIN departments  
ON employees.department_id;
```

Reality

```
SELECT * FROM employees LEFT JOIN departments  
ON employees.employee_id = departments.employee_id  
UNION  
SELECT * FROM employees RIGHT JOIN departments  
ON employees.employee_id = departments.employee_id;
```

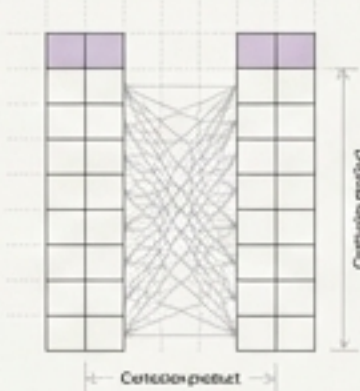
Expanding possibilities with advanced joins

CROSS JOIN

Combines every row from the first table with every row from the second table (Cartesian product).

Blueprint

```
SELECT *  
FROM table1  
CROSS JOIN table2;
```



Reality

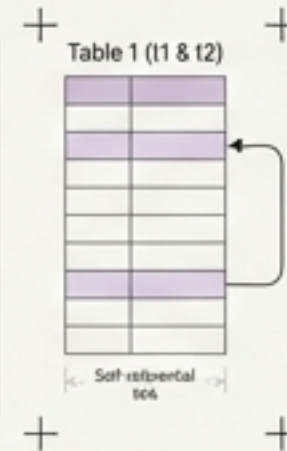
```
SELECT *  
FROM employees  
CROSS JOIN  
departments;
```

SELF JOIN

Joins a table with itself.

Blueprint

```
SELECT *  
FROM table1 t1, table1 t2  
WHERE  
t1.column = t2.column;
```



Reality

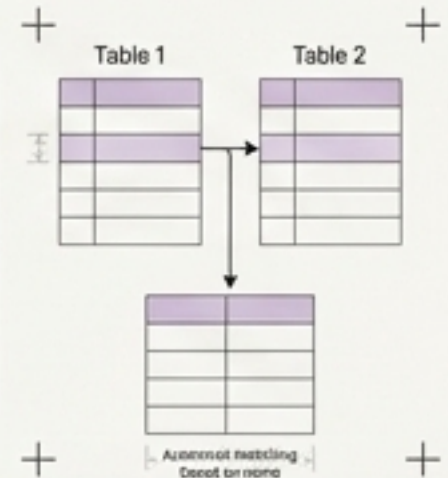
```
SELECT *  
FROM employees t1, employees t2  
WHERE  
t1.employee_id = t2.employee_id;
```

NATURAL JOIN

Matches columns with the same name in both tables automatically.

Blueprint

```
SELECT *  
FROM table1  
NATURAL JOIN table2;
```



Reality

```
SELECT *  
FROM employees  
NATURAL JOIN  
departments;
```

Running queries within queries

IN

Determines whether a value matches any value in a subquery result. It is often used in the WHERE clause.

Blueprint

```
SELECT column(s) FROM table  
WHERE value IN (subquery);
```

Reality

```
SELECT * FROM customers  
WHERE city IN  
(SELECT city FROM suppliers);
```

Dynamic comparisons using advanced subqueries

ANY

Compares a value to any value returned by a subquery. Used with comparison operators (=, >, <, etc).

Blueprint

```
SELECT column(s) FROM table  
WHERE value < ANY (subquery);
```

Reality

```
SELECT * FROM products  
WHERE price < ANY  
  (SELECT unit_price FROM supplier_products);
```

ALL

Compares a value to all values returned by a subquery. Used with comparison operators (=, >, <, etc).

Blueprint

```
SELECT column(s) FROM table  
WHERE value > ALL (subquery);
```

Reality

```
SELECT * FROM orders  
WHERE order_amount > ALL  
  (SELECT total_amount FROM previous_orders);
```

Keep learning and keep exploring

Data Definition Language (DDL)

CREATE, ALTER, DROP, TRUNCATE

Data Manipulation Language (DML)

INSERT, UPDATE, DELETE

Querying Data

SELECT, WHERE, ORDER BY

Aggregate Functions

GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX

Joining Commands

INNER, OUTER, CROSS, SELF, NATURAL

Subqueries

IN, ANY, ALL